28. NLP - Natural Language Processing

Metni veriye, veriyi modele dönüştürmek

Text Cleaning • Tokenization • TF-IDF • Embeddings • Classification • Topic Modeling



Bu modülün amacı

Doğal dil verisini yalnızca kelime sayımı olarak değil; preprocessing, representation, modeling, evaluation ve hata analizi zinciriyle uçtan uca işleyebilmek.

Kazanım	Modül sonunda
Problem framing	Text classification, sentiment, similarity, search ve extraction görevlerini ayırabilirsin.
Preprocessing	Tokenization, normalization, stopword, stemming/lemmatization kararlarını bağlama göre verebilirsin.
Representation	Bag-of-Words, n-gram, TF-IDF ve embeddings arasındaki farkı anlayabilirsin.
Modeling	Linear modeller, Naive Bayes ve embedding tabanlı modeller için pipeline kurabilirsin.
Evaluation	F1, PR-AUC, macro-F1 ve error analysis ile text modellerini değerlendirebilirsin.
Modern NLP köprüsü	Transformers, contextual embeddings ve LLM yaklaşımına hazır hale gelirsin.

1. NLP nedir ve hangi problemleri çözer?

NLP (Natural Language Processing), insan dilini makinelerin işleyebileceği temsillere dönüştürme alanıdır. Veri bilimci için temel hedef, metinden ölçülebilir sinyal çıkarmaktır.

Görev	Örnek
Text classification	Spam / değil, şikayet kategorisi, ticket routing
Sentiment analysis	Olumlu / olumsuz müşteri yorumu
Named Entity Recognition	Kişi, şirket, ürün, tarih çıkarımı
Similarity / semantic search	Benzer belge veya soru bulma
Topic modeling	Belge koleksiyonundaki temaları keşfetme
Summarization	Uzun metni kısa özete dönüştürme
Question answering / RAG	Belge koleksiyonundan bilgi getirip cevap üretme

> **NLP problem framing**
> Önce “metinden ne karar üretilecek?” sorusunu cevapla. Aynı review metni sentiment, konu sınıflama, embedding search veya özetleme için farklı pipeline gerektirir.

2. Text veri neden zordur?

- Değişken uzunluk
- Yüksek boyut ve sparse representation
- Eş anlam, çok anlamlılık ve bağlam
- İmla, emoji, kısaltma ve dil karışımı
- Kelime sırasının anlam taşıması
- Distribution shift: yeni ürünler, yeni jargon, yeni kullanıcı dili

3. NLP pipeline



Klasik NLP pipeline çoğunlukla dört ana katmandır: veri hazırlama, metin temsili, model ve evaluation. Modern transformer sistemlerinde bazı katmanlar modelin içinde öğrenilir ancak problem mantığı aynı kalır.

Katman	Soru
Cleaning	Gürültü mü, yoksa anlam taşıyan bilgi mi siliyorum?
Tokenization	Metni hangi birimlere ayırıyorum?
Representation	Kelimeleri / cümleleri nasıl sayısallaştırıyorum?
Model	Hangi görev için hangi inductive bias gerekli?
Evaluation	Metric gerçek iş amacını temsil ediyor mu?
Error analysis	Model hangi dil kalıplarında yanılıyor?

4. Normalization ve cleaning

Temizleme her projede aynı değildir. Örneğin sentiment analizinde “!!!”, emoji ve büyük harfler önemli sinyal olabilir; onları körlemesine silmek hatadır.

```
import re

def normalize_text(text):
    text = text.strip()
    text = re.sub(r"\s+", " ", text)
    return text
```

> **Altın kural**
> Temizleme adımının amacı “metni güzelleştirmek” değil, görev için gereksiz varyasyonu azaltmaktır.

5. Tokenization

Tokenization metni kelime, alt-kelime veya karakter parçalarına ayırır. Klasik modeller kelime tokenlarıyla, modern transformers ise çoğunlukla subword tokenization ile çalışır.

Yaklaşım	Artı	Eksi
Whitespace / word	Basit ve hızlı	Noktalama, birleşik kelime sorunları
Regex tokenizer	Kontrollü	Dil bağımlı kurallar gerekir
Character	OOV sorunu az	Uzun sequence ve zayıf semantik
Subword (BPE/WordPiece)	OOV azaltır, morphology yakalar	Daha karmaşık

```
text = "Müşteri hizmetleri çok hızlıydı!"
tokens = text.lower().split()
print(tokens)
```

6. Stopwords

“ve”, “bir”, “ile” gibi sık kelimeleri çıkarmak bazı klasik bilgi erişim görevlerinde işe yarayabilir. Ancak sentiment, intent veya transformer modellerinde körlemesine stopword kaldırmak anlam kaybı yaratabilir.

> **Örnek**
> “ürün iyi değil” cümlesinde “değil” kelimesini stopword sanıp silersen sentiment tersine döner.

7. Stemming vs Lemmatization

Yöntem	Mantık	Not
Stemming	Kelimeleri kaba köke kırpar	Hızlı ama anlamsız kök üretebilir
Lemmatization	Sözlük/gramer ile lemma bulur	Daha temiz ama dil kaynaklarına bağlı

8. Bag of Words

Bag of Words, belgeyi kelime frekanslarından oluşan vektör olarak temsil eder. Kelime sırasını büyük ölçüde kaybeder ama güçlü bir baseline'dır.

Belge	iyi	hızlı	kargo	kötü
D1: iyi hızlı kargo	1	1	1	0
D2: kötü kargo	0	0	1	1
D3: iyi iyi	2	0	0	0

9. N-gram

N-gram ardışık n token grubudur. Bigram, “iyi değil” gibi tek kelimelerin yakalayamadığı yerel bağlamı korur.

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer(
    ngram_range=(1, 2),
    min_df=3
)
X = vec.fit_transform(texts)
```

> **Trade-off**
> N-gram arttıkça vocabulary ve sparsity hızla büyür. Daha çok bağlam kazanırken hesap maliyeti ve overfitting riski artar.

10. TF-IDF

Document-Term Matrix

	iyi	hızlı	kötü	ürün	kargo
D1: iyi hızlı kargo	1.0	1.4	0	0	1.4
D2: kötü ürün	0	0	1.4	1.0	0
D3: iyi ürün iyi	2.0	0	0	1.0	0

TF-IDF bir kelimeyi yalnızca belge içinde sık olduğu için değil, tüm koleksiyonda ne kadar ayırt edici olduğuna göre ağırlıklandırır.

```
TF-IDF(t,d) = TF(t,d) × IDF(t)
```

Yüksek ağırlık: belgede önemli, corpus genelinde nispeten nadir terim.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.95,
    sublinear_tf=True
)
X = vectorizer.fit_transform(train_text)
```

Güçlü baseline
TF-IDF + Logistic Regression / Linear SVM birçok text classification probleminde şaşırtıcı derecede güçlü ve hızlı baseline'dır.

11. Sparse matrix ve vocabulary yönetimi

TF-IDF matrisi çoğunlukla çok büyük ama sparse'dır. Bu yüzden dense'e çevirmek bellek patlamasına yol açabilir.

Parametre	Etkisi
min_df	Çok nadir kelimeleri atar
max_df	Aşırı sık / ayırt etmeyen terimleri atar
max_features	Vocabulary üst sınırı
ngram_range	Yerel bağlam miktarı
analyzer="char"	Karakter n-gram; typo ve morphology için güçlü olabilir

```
# Yanlış olabilir:
# X_dense = X.toarray()

print(X.shape)
print(X.nnz)   # non-zero count
```

12. Character n-grams

Ürün adı, kullanıcı adı, typo, farklı ekler veya kısa metinlerde character n-gram güçlü olabilir.

```
TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(3, 5)
)
```

> **Türkçe açısından**
> Eklemeli dillerde character/subword yaklaşımı kelime varyasyonlarını klasik word tokenization'dan daha iyi yakalayabilir.

13. Klasik text classification modelleri

Model	Güçlü yön	Not
Logistic Regression	Hızlı, yorumlanabilir, sparse uyumlu	TF-IDF için güçlü default
Linear SVM	Yüksek boyutta güçlü margin	Probability doğal değildir
Multinomial Naive Bayes	Çok hızlı, word-count tabanlı	Güçlü baseline
Tree/Boosting	Dense aggregate/text metadata ile iyi	Raw sparse TF-IDF için her zaman ideal değil

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=3)),
    ("clf", LogisticRegression(max_iter=2000))
])

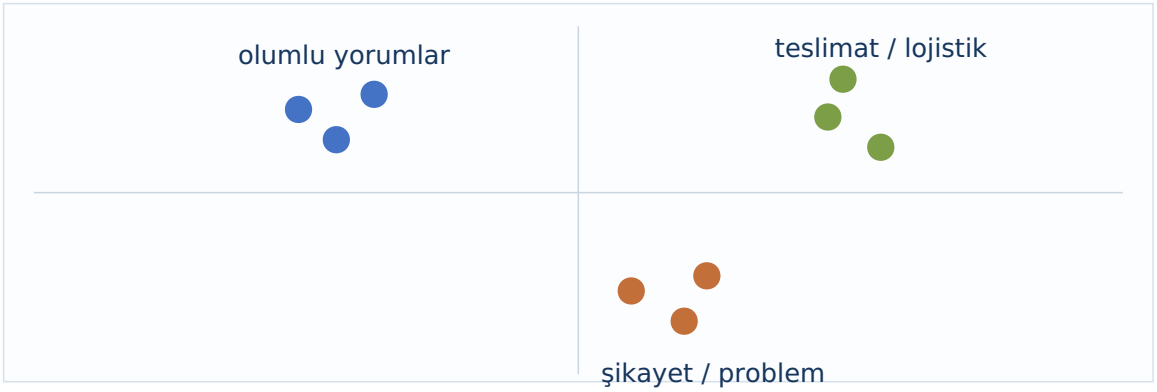
pipe.fit(train_text, y_train)
```

14. Multinomial Naive Bayes

Class verildiğinde kelime feature'larının koşullu bağımsızlığını varsayar. Varsayım basit olsa da özellikle spam/topic classification gibi görevlerde hızlı baseline'dır.

> **Baseline stratejisi**
> Önce basit ve güçlü baseline kur. Transformer'a yalnız problem/metric gerçekten gerektiriyorsa geç.

15. Word embeddings



Semantik olarak benzer metinler vektör uzayında yakınlaşır

Embedding her kelimeyi dense bir vektör olarak temsil eder. Vektör geometrisi semantik benzerliği taşıyabilir.

Yaklaşım	Özellik
Word2Vec	Local context üzerinden kelime vektörü öğrenir
GloVe	Global co-occurrence istatistikleri
FastText	Subword parçaları; morphology/OOV için güçlü
Contextual embedding	Kelime anlamı cümle bağlamına göre değişir

> **Statik embedding sınırı**
> “yüz” kelimesi “insan yüzü” ve “yüz metre” bağlamlarında aynı Word2Vec vektörüne sahiptir. Contextual modeller bunu ayırabilir.

16. Cümle / belge embeddings

Bir cümleyi veya belgeyi tek dense vektöre dönüştürmek semantic search, clustering, duplicate detection ve RAG retrieval için temel yapıdır.

17. Cosine similarity

```
cosine(a,b) = (a · b) / (||a|| ||b||)
```

Embedding vektörleri arasında yön benzerliğini ölçer. Semantic search'te çok yaygındır.

```python
import numpy as np

def cosine(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

18. Sentiment Analysis

Sentiment sınıflama, metindeki duygu/yorum polaritesini tahmin eder. Ürün review, sosyal medya veya çağrı merkezi transkriptlerinde kullanılır.

Risk	Örnek
Negation	“kötü değil”
Irony / sarcasm	“harika, yine çöktü”
Domain language	“agresif fiyat” finans vs günlük dil
Class imbalance	Olumsuz yorum az olabilir
Label ambiguity	3 yıldız gerçekten neutral mı?

> **Evaluation**
> Sentiment modelinde yalnız accuracy değil class bazlı precision/recall ve confusion matrix incele.

19. Text classification evaluation

Metric	Ne zaman?
Accuracy	Sınıflar dengeli ve hata maliyeti benzerse
Macro F1	Her sınıf eşit önemliyse
Weighted F1	Class support'a göre ağırlıklı özet
PR-AUC	Rare positive sınıf
Confusion Matrix	Hangi etiketler karışıyor?

20. Error analysis

1. False positive / false negative metinleri örnekle.
2. Uzunluk, dil, typo, emoji, domain jargon segmentlerine ayır.
3. Label noise var mı kontrol et.
4. Modelin hangi kelime/n-gram kalıplarına aşırı güvendiğini incele.
5. Yeni preprocessing veya feature hipotezi üret.

> **Text projelerinde kritik**
> Model hatalarının önemli kısmı modelden değil label tanımı, annotation kalitesi ve domain shift'ten kaynaklanabilir.

21. Data leakage NLP'de

Leakage	Örnek
Duplicate leakage	Aynı review train ve test'te
User leakage	Aynı kullanıcı yazım stili train/test
Temporal leakage	Gelecekte çıkan ürün isimleri train'e sızıyor
Preprocessing leakage	TF-IDF vocabulary tüm dataset'e fit
Label proxy	Etiket bilgisi metne sistem tarafından eklenmiş

22. Topic Modeling

Topic modeling etiket olmadan belge koleksiyonundaki gizli temaları keşfetmeye çalışır.

Yöntem	Sezgi
LDA	Belge = topic karışımı, topic = kelime dağılımı
NMF	Non-negative matrix factorization ile topic bileşenleri
Embedding + clustering	Semantic vector uzayında kümeler

> **Topic ≠ gerçek kategori**
> Topic model çıktısı keşif aracıdır. İş anlamı için topic kelimelerini ve örnek belgeleri insan gözüyle incelemek gerekir.

23. LDA pipeline

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

cv = CountVectorizer(min_df=5, max_df=0.9)
X = cv.fit_transform(texts)

lda = LatentDirichletAllocation(
    n_components=8,
    random_state=42
)
Z = lda.fit_transform(X)
```

24. Topic sayısı seçimi

Tek doğru k yoktur. Coherence, stability ve en önemlisi iş yorumlanabilirliği birlikte değerlendirilir.

25. Text clustering ve semantic search

Sentence/document embeddings + cosine similarity, semantic search ve document clustering için modern standarda yakındır.

Adım	İşlem
1	Metni embedding vektörüne dönüştür
2	Vektörleri normalize et
3	Cosine / dot product ile benzerlik hesapla
4	Approximate nearest neighbor index kullan
5	Metadata filter / reranking ekle

> **RAG bağlantısı**
> RAG sisteminin retrieval katmanı temelde document chunk -> embedding -> vector search zinciridir.

26. Duplicate / near-duplicate detection

Exact string matching yerine embedding/cosine veya MinHash gibi yöntemlerle semantik/lexical benzer kopyalar bulunabilir.

27. Named Entity Recognition (NER)

Metinden kişi, kurum, tarih, ürün veya lokasyon span'ları çıkarmak için kullanılır.

Entity	Örnek
PERSON	Ayşe Yılmaz
ORG	OpenAI
DATE	10 Ekim 2026
PRODUCT	iPhone 18

28. Sequence labeling ve information extraction

NER, POS tagging ve slot filling gibi görevlerde her token için etiket üretilir. Bazen BIO formatı kullanılır.

Token	Etiket
OpenAI	B-ORG
İstanbul	B-LOC
ofisini	O
açtı	O

29. Kural tabanlı + ML hibrit sistem

Regex, sözlük ve ML birlikte kullanılabilir. Özellikle fatura, ticket veya belirli formatlı domainlerde saf ML yerine hibrit yaklaşım daha güvenilir olabilir.

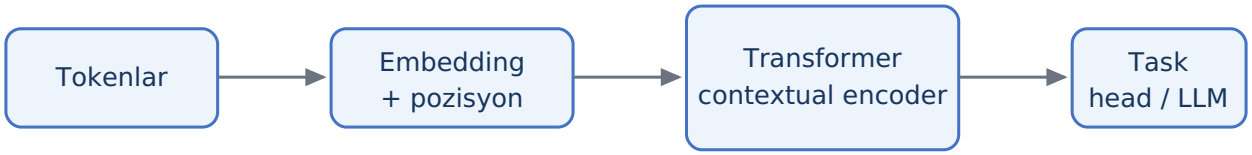
> **Senior tasarım**
> Her NLP probleminde LLM kullanmak zorunda değilsin. Regex + TF-IDF + linear model üretimde bazen daha hızlı, ucuz ve denetlenebilirdir.

30. Text augmentation

- Synonym replacement
- Back translation
- Paraphrasing
- Noise / typo injection

> **Dikkat**
> Augmentation label semantiğini bozabilir. Özellikle sentiment ve intent task'lerinde otomatik augmentation validasyonla doğrulanmalıdır.

31. Transformer ve LLM'e geçiş



Transformers kelimeleri bağlam içinde temsil eder. Self-attention sayesinde tokenlar birbirleriyle doğrudan etkileşir; aynı kelime farklı cümlede farklı contextual embedding üretebilir.

Klasik NLP	Transformer NLP
TF-IDF sparse vectors	Contextual dense representations
Feature engineering yoğun	Representation büyük ölçüde model tarafından öğrenilir
Linear models hızlı	Fine-tuning/inference daha pahalı
Local n-gram context	Uzun menzilli context yakalanabilir

> **Ne zaman transformer?**
> Klasik baseline anlamlı performans sınırına geliyorsa, semantic/contextual ihtiyaç yüksekse veya transfer learning avantajı önemliyse.

32. Fine-tuning vs embeddings vs prompting

Yaklaşım	Kullanım
Fine-tuning	Task-specific model ağırlıklarını güncelle
Embeddings	Search, clustering, retrieval, similarity
Prompting	Genel LLM'i instruction ile yönlendir
RAG	External knowledge retrieval + generation

33. Gerçek vaka: müşteri yorumlarından sentiment

Aşama	Karar
Target	positive / neutral / negative
Split	Zamana veya ürüne göre leakage-safe
Baseline	TF-IDF + Logistic Regression
Metric	Macro F1 + class recall
Error analysis	Negation, sarcasm, kısa review, emoji
Advanced	Sentence transformer / fine-tuned encoder

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer(
        ngram_range=(1,2),
        min_df=3,
        sublinear_tf=True
    )),
    ("clf", LogisticRegression(
        class_weight="balanced",
        max_iter=2000
    ))
])
```

> **Portföy farkı**
> Sadece model skoru değil; error taxonomy, label quality analizi ve baseline-vs-transformer karşılaştırması göster.

34. Gerçek vaka: support ticket routing

Amaç gelen ticket'ı Billing / Delivery / Technical / Cancellation gibi ekiplere otomatik yönlendirmek.

Problem noktası	Çözüm
Class imbalance	Macro F1 + class weights
Yeni kategori	Unknown / fallback akışı
Kısa metin	Character n-gram veya embedding
Dil karışımı	Multilingual model veya language routing
Yanlış route maliyeti	Top-2 suggestion / human-in-the-loop

35. Gerçek vaka: semantic FAQ search

Kullanıcı sorusunu FAQ embedding'leriyle eşleştirip en yakın cevabı bul.

1. FAQ soru/cevapları chunk'la.
2. Embedding üret.
3. Vector index oluştur.
4. Query embedding al.
5. Top-k similarity getir.
6. Threshold altında ise “bulamadım” fallback'i ver.

> **Bu vaka sonraki modüllere köprü**
> Vector Databases ve RAG modüllerinin çekirdeği burada başlar.

36. Production NLP checklist

- Text encoding ve Unicode normalize mi?
- Dil tespiti gerekiyor mu?
- PII / kişisel veri masking var mı?
- Vocabulary / tokenizer versionlandı mı?
- Train-serving preprocessing aynı mı?
- Max token / truncation davranışı tanımlı mı?
- Latency ve batch size ölçüldü mü?
- Drift: yeni jargon / yeni ürün / yeni dil izleniyor mu?
- Confidence / abstention / fallback var mı?
- Human review feedback loop var mı?

37. Drift NLP'de

Drift	Örnek
Vocabulary drift	Yeni ürün / yeni argo
Topic drift	Yeni müşteri problemi
Label policy drift	Ticket kategorisi yeniden tanımlandı
Language mix drift	İngilizce/Türkçe oranı değişti

> **Monitoring**
> OOV rate, embedding distribution, class distribution, confidence ve error sample monitoring yararlı olabilir.

38. Mülakat soruları

Soru	Kısa cevap
Bag of Words nedir?	Belgeyi token count vektörüyle temsil eder.
TF-IDF neden?	Sık ama ayırt edici olmayan terimlerin ağırlığını düşürür.
N-gram ne sağlar?	Yerel kelime sırası ve phrase bilgisi.
Stemming vs lemmatization?	Kaba kök kırpma vs dil bilgili lemma.
Stopword her zaman silinir mi?	Hayır; task semantiğini bozabilir.
Sparse matrix neden?	Vocabulary büyük, çoğu belge çoğu kelimeyi içermez.
Naive Bayes neden güçlü baseline?	Hızlı ve word-count yapısına uyumlu.
Embedding nedir?	Metni dense semantik vektör olarak temsil eder.
Cosine similarity?	Vektör yön benzerliği.
Macro F1 ne zaman?	Sınıflar eşit önemli / imbalance varsa.
Data leakage NLP örneği?	TF-IDF vocabulary tüm datasete fit etmek veya duplicate train-test.
Topic modeling supervised mı?	Hayır, çoğunlukla unsupervised.
Transformer farkı?	Contextual token representations ve attention.
RAG ile NLP ilişkisi?	Retrieval için chunking/embedding/vector search NLP temellerini kullanır.

39. Uygulama alıştırmaları

1. IMDB/Türkçe review datasında TF-IDF + Logistic Regression sentiment modeli kur.
2. Word unigram vs (1,2)-gram Macro F1 karşılaştır.
3. Word vs character TF-IDF karşılaştır.
4. Stopword kaldırmanın score'a etkisini deneyle.
5. MultinomialNB ile Logistic Regression baseline karşılaştır.
6. Confusion matrix'ten 50 hata örneği için error taxonomy oluştur.
7. Embedding + cosine ile semantic duplicate finder yap.
8. K-Means ile document embeddings üzerinde topic-like cluster oluştur.
9. LDA ile 8 topic çıkar ve insan gözüyle isimlendir.
10. Support ticket datasetinde time-based split uygula.

40. Mini portföy projesi

Proje: Customer Review Intelligence Platform

Çıktı	İçerik
01_eda.ipynb	Language, length, labels, duplicates
02_tfidf_baseline.ipynb	Word/char TF-IDF + linear models
03_error_analysis.ipynb	Negation, typo, short-text hataları
04_embeddings.ipynb	Semantic search + clustering
05_topic_modeling.ipynb	LDA / embedding clusters
src/pipeline.py	Reusable preprocessing + model
report.md	Metric, failure modes, business recommendation

CV cümlesi
Built an end-to-end NLP review intelligence system combining TF-IDF classification, semantic embeddings, topic discovery and systematic error analysis.

41. Final cheat sheet

Kavram	Hatırlanacak fikir
Tokenization	Metni işlenebilir birimlere ayır
Normalization	Gereksiz varyasyonu azalt, sinyali silme
BoW	Token count representation
N-gram	Yerel bağlam
TF-IDF	Ayırt edici terim ağırlığı
Sparse matrix	Yüksek boyut, çoğu değer 0
Naive Bayes	Hızlı probabilistic baseline
Linear SVM / Logistic	Sparse text için güçlü baseline
Embedding	Dense semantic representation
Cosine	Semantic vector benzerliği
Topic modeling	Unsupervised tema keşfi
NER	Entity span çıkarımı
Macro F1	Imbalanced multiclass için güçlü özet
Transformer	Contextual representation
RAG köprüsü	Chunking + embedding + retrieval

Bu modülden sonra
Deep Learning ve PyTorch modülleriyle text representation öğrenimini modelin kendisine bırakmaya başlayacak, Transformers & LLM bölümünde contextual NLP yaklaşımına geçeceksin.

Modül 28 tamamlandı.